

GOLANG

Living Textbook cho Software Engineering và DevOps/SRE

Edition nền móng

Được kiểm chứng với Go 1.27.1 - 20-09-2026

Markdown là nguồn gốc. PDF là bản đọc được, có thể tái tạo cục bộ.

Mục lục của edition này

Đây là edition nền móng. Mục lục tổng thể và thứ tự các phần tiếp theo được giữ trong `book/README.md` để có thể mở rộng mà không giả vờ rằng chúng đã được viết xong.

- Lời nói đầu
- Chương 1. Go toolchain và vòng lặp có thể kiểm chứng
- Chương 2. Một chương trình Go bắt đầu từ package

Lời nói đầu

Cuốn sách này dùng để làm gì?

Anh không cần thêm một danh sách cú pháp để chép lại. Điều khó hơn, và đáng học hơn, là tự đi từ một yêu cầu mơ hồ đến một chương trình có thể giải thích, kiểm thử, sửa và vận hành. Go phù hợp với hành trình ấy vì ngôn ngữ nhỏ vừa đủ để ta nhìn rõ phần lớn quyết định, còn thư viện chuẩn lại chạm được vào file, network, process và dịch vụ thật.

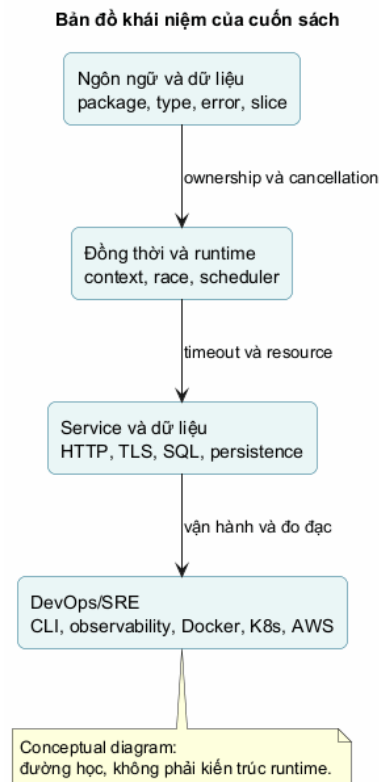
Em sẽ không coi việc đọc xong một chương là bằng chứng anh đã biết. Sau mỗi khái niệm sẽ có lúc phải đoán kết quả, lần theo dữ liệu, đọc code lạ hoặc sửa một lỗi có chủ đích. Nếu bí, hãy ghi giả thuyết trước khi chạy code. Một lần đoán sai nhưng biết vì sao sai có giá trị hơn một lần nhận đáp án đúng từ AI.

Edition này được kiểm chứng với **Go 1.27.1** vào **20-09-2026**. Những phần gắn với release hoặc implementation sẽ được gắn nguồn và mô tả đúng mức của chúng: language specification không phải runtime implementation; một kết quả benchmark trên máy em cũng không phải định luật tự nhiên.

Cách dùng sách

Đọc phần giải thích, gõ lại ví dụ tối thiểu, rồi thay đổi một điều nhỏ trước khi xem lời giải. Hãy để terminal phản biện suy nghĩ của mình. `go fmt`, `go test` và `go vet` không phải nghi thức; chúng là ba câu hỏi khác nhau: code có được trình bày theo chuẩn chung không, hành vi có giữ lời hứa không, và có lỗi tĩnh đáng ngờ không?

Khi đi tới các phần DevOps/SRE, em sẽ luôn quay lại nền tảng. Một timeout HTTP không chỉ là một con số; nó nối với `context`, socket, pool kết nối, retry và ngân sách thời gian của request. Sơ đồ sau là bản đồ khái niệm của đường đi đó, không phải một guarantee của Go runtime.



Quy ước

Code block là code có thể gõ, trừ khi ghi rõ “minh họa”. Output terminal chỉ được nêu khi có thể chạy hoặc khi được gắn nhãn minh họa. Với bài tập, dừng ở đường ngăn trước khi đọc đáp án; đừng biến em thành nút autocomplete có chân.

Nhiệm đọc của edition này

Thành phần	Mục tiêu đọc
Văn xuôi	Hiểu lập luận dài mà không phải nheo mắt trên laptop.
Code	Phân biệt rõ với lời văn, đủ lớn để đối chiếu từng dòng.
Sơ đồ	Làm rõ quan hệ; luôn ghi khi chỉ là conceptual diagram.
Bài tập và đáp án	Buộc dự đoán trước, rồi giải thích nguyên nhân sau.

Chương 1. Go toolchain và vòng lặp có thể kiểm chứng

Go không chỉ là compiler. Khi anh cài Go, thứ xuất hiện trước mặt là lệnh `go`, một cửa vào nhiều công cụ cùng chia sẻ hiểu biết về module và package. Nó có thể chạy chương trình, build binary, format code, chạy test, tải dependency và phân tích một phần lỗi tĩnh. Việc có một command thống nhất làm giảm số mảnh ghép phải nhớ, nhưng không thay thế việc hiểu từng mảnh đang làm gì.

Phiên bản dùng trong edition này là Go 1.27.1. “Mới nhất” không có nghĩa là chạy theo mọi bản thử nghiệm; nó là stable release mà tài liệu chính thức đang phát hành. Mỗi ví dụ có thể dùng ngôn ngữ hiện hành, nhưng project thật vẫn cần chọn go directive theo compatibility policy của chính nó. Từ Go 1.26, `go mod init` có chủ ý tạo directive thấp hơn toolchain một major-minor để khuyến khích tương thích với các bản còn được hỗ trợ. Vì vậy ta sẽ xem `go.mod` là một quyết định tương thích, không phải nhãn trang trí.

Workspace không phải GOPATH cũ

Ngày trước, nhiều Go project bị buộc nằm dưới `GOPATH/src`. Với modules, source có thể nằm ở nơi hợp lý cho project. Một module là đơn vị quản lý dependency; file `go.mod` đặt ranh giới của nó và ghi module path cùng Go version. Package là đơn vị code/import nhỏ hơn: một thư mục thường tạo ra một package, và nhiều package có thể thuộc cùng module.

Sự phân biệt này cứu anh khỏi một lỗi tư duy phổ biến: thấy tên thư mục rồi đoán đó là import path. Import path được xác định từ module path cộng vị trí thư mục; nó không tự sinh từ ổ đĩa. Đó là lý do một project clone từ GitHub có thể build ở máy khác mà không cần cùng tên người dùng Windows.

<code>module example.com/opsprobe</code>	<code><- go.mod xác định gốc module</code>
<code>├─ cmd/opsprobe</code>	<code><- package main, tạo executable</code>
<code>├─ internal/check</code>	<code><- package chỉ module này được import</code>
<code>└─ go.mod</code>	

`go env GOROOT GOPATH GOMOD` giúp kiểm tra ba điều khác nhau. `GOROOT` là nơi toolchain nằm; `GOPATH` vẫn có ích làm cache và nơi cài một số binary; `GOMOD` cho biết lệnh đang thấy file module nào. Nếu `GOMOD` là `/dev/null` hoặc đường dẫn không như dự kiến, đừng vội xoá cache: kiểm tra thư mục hiện tại trước.

Vòng lặp nhỏ nhưng nghiêm túc

Một thay đổi nhỏ nên đi qua vòng lặp sau. Nó là conceptual diagram, không phải một protocol cứng: với một thử nghiệm một dòng, ta có thể dừng ở bước chạy; với code gửi vào production, thêm test tích hợp và review.



1. Viết một giả thuyết và thay đổi nhỏ nhất có thể kiểm chứng nó.
2. Chạy `go fmt ./...` để bỏ tranh luận vô ích về khoảng trắng.
3. Chạy `go test ./...`; không có file test vẫn là thông tin, không phải thành công giả tạo.
4. Chạy `go vet ./...` khi module đủ hoàn chỉnh để phân tích.
5. Đọc output, sửa nguyên nhân rồi mới lặp lại.

Điểm dễ bị AI che khuất là bước 1. Trước khi hỏi một công cụ, hãy nói được “em tin lỗi nằm ở đâu và dấu hiệu nào sẽ bác bỏ điều đó”. Điều này biến terminal từ máy phát thông báo thành một dụng cụ đo.

Lab: kiểm tra môi trường mà không đoán

Tạo một thư mục rỗng và chạy:

```
go version
go env GOROOT GOPATH GOMOD
go help modules
```

Đừng cố diễn giải tất cả output ngay. Hãy trả lời ba câu: toolchain nào đang chạy, cache/user workspace nào đang được dùng, và thư mục hiện tại đã nằm trong một module chưa. Khi ba câu này rõ, rất nhiều lỗi “Go không tìm thấy package” trở nên có hình dạng cụ thể.

Bài tập

Bạn A đang đứng trong `D:\work\weather` và `go env GOMOD` cho `/dev/null`. Bạn ấy kết luận Go hỏng vì project chưa có `go.mod`. Kết luận đó thiếu điều gì?

ĐÁP ÁN - chỉ đọc sau khi đã tự làm

`/dev/null` thường chỉ cho thấy command hiện tại không thấy module file ở thư mục đang đứng hay thư mục cha. Nó chưa chứng minh Go hỏng. A cần kiểm tra current directory, xác nhận project có thật sự đã `go mod init`, hoặc đi vào đúng thư mục gốc. Chỉ sau đó mới xét tới `GOWORK`, environment override hay toolchain.

Tóm lại

Module là ranh giới dependency; package là ranh giới code/import. Hãy quan sát `GOMOD` trước khi sửa, và dùng vòng lặp viết - format - test - vet để tạo phản hồi ngắn. Chương sau sẽ đặt một chương trình đầu tiên vào đúng các ranh giới đó, thay vì ném một file `main.go` vào bất kỳ thư mục nào rồi cầu may.

Nguồn

- Go Modules Reference, go.dev/ref/mod.
- Go 1.26 Release Notes, phần `go mod init`, go.dev/doc/go1.26.
- Go Command documentation, go.dev/cmd/go.

Chương 2. Một chương trình Go bắt đầu từ package

Một file Go không tự nhiên trở thành một chương trình chỉ vì có `func main`. Nó trước hết thuộc về một package. Với executable, package đó phải tên `main` và cần một hàm `main` không tham số, không return. Compiler dùng quy ước này để biết điểm bắt đầu của binary; nó không phải lời mời gọi nhét toàn bộ project vào một file.

Tạo module nhỏ

Trong một thư mục mới, chạy `go mod init example.com/hello`. Module path ở đây là định danh logic; nếu sau này publish code, nó thường trùng địa chỉ repository để người khác import được. Với bài lab cục bộ, `example.com` là domain được dành cho ví dụ, nên không giả vờ rằng ta sở hữu một domain thật.

Tạo `main.go`:

```
package main

import "fmt"

func main() {
    name := "Anh"
    fmt.Printf("Chào %s, hãy thay đổi một dòng rồi chạy lại.\n", name)
}
```

`:=` vừa khai báo biến mới vừa để compiler suy luận kiểu từ biểu thức bên phải. Ở đây `name` có kiểu `string`. Suy luận kiểu không có nghĩa Go là dynamically typed: kiểu vẫn được xác định khi compile, và việc gán sau đó phải phù hợp. Nếu anh viết `name = 42`, compiler dừng trước khi binary tồn tại. Đây là một lỗi rề: phát hiện lúc build thường rề hơn phát hiện lúc service đang phục vụ người dùng.

Chạy `go run ..` Dấu chấm nghĩa là package trong thư mục hiện tại, chú không phải “chạy file vừa mở trong editor”. Thói quen này quan trọng ngay khi package có nhiều file hoặc build tags. `go run` build một executable tạm

rồi chạy nó; để tạo artifact có thể đưa cho người khác, dùng `go build -o bin/hello ..`

Đọc lỗi compiler như một tọa độ

Hãy cố ý đổi `import` thành `"fmtt"` rồi chạy lại. Error message có thể trông khó chịu, nhưng nó thường nói ba thứ hữu ích: package hoặc file nào lỗi, dòng nào, và compiler đang thiếu điều gì. Đừng sửa mù theo phần cuối message. Đọc từ dòng đầu liên quan source, sửa lỗi gốc, rồi chạy lại; một lỗi `import` có thể kéo theo nhiều thông báo phụ.

Một lỗi khác hay gặp là `import fmt` rồi không dùng nó. Go coi `unused import` là error vì `import` có thể tạo dependency và tác dụng khởi tạo; giữ `import` chết làm đọc code khó hơn. Đây không phải compiler “khó tính cho vui”, mà là một lựa chọn để dependency nhìn thấy được còn source giữ sạch.

Zero value là một thiết kế tool

Khai báo `var retries int` mà chưa gán cho nó vẫn hợp lệ: giá trị ban đầu của `int` là `0`. `bool` là `false`, `string` là chuỗi rỗng, `pointer/map/slice/channel/ function/interface` là `nil`. Zero value giúp type có thể dùng được ngay mà không cần constructor khi điều đó hợp lý, nhưng không biến `nil` thành an toàn. Việc ghi vào `nil map` sẽ panic; đọc `nil map` thì hợp lệ và trả zero value của element. Ta sẽ quay lại chi tiết aliasing và `nil` ở chương dữ liệu.

```
var attempts int
var ready bool

fmt.Println(attempts, ready) // 0 false
```

Đoạn trên in output xác định. Nếu ví dụ sau này phụ thuộc thời gian, `network` hoặc `scheduler`, sách sẽ nói rõ điều đó thay vì hứa hẹn một output đẹp nhưng không lặp lại được.

Điểm nối sang DevOps/SRE

Một CLI health check nhỏ hay exporter lớn đều bắt đầu từ package main, nhưng đa số logic không nên sống ở đó. main thường chỉ wiring: đọc config, tạo dependency, gọi luồng chính và quyết định exit code. Logic kiểm tra HTTP, parse file hay tính retry nên ở package riêng để test không cần fork process. Chưa cần xây kiến trúc ngay hôm nay; chỉ cần thấy rằng cách ta khởi động chương trình đã ảnh hưởng trực tiếp tới cách ta kiểm thử và vận hành nó sau này.

Bài tập

Không chạy code trước. Dự đoán: chương trình dưới đây compile hay không? Nếu compile, nó in gì? Sau đó hãy tạo module và kiểm tra dự đoán.

```
package main

import "fmt"

func main() {
    var port int
    port = port + 8080
    fmt.Println(port)
}
```

ĐÁP ÁN - chỉ đọc sau khi đã tự làm

Nó compile và in 8080. port nhận zero value 0, rồi phép cộng tạo giá trị mới để gán lại. Cạm bẫy là nhầm zero value với “chưa có giá trị” trong mọi miền nghiệp vụ. Với port, 0 còn có nghĩa đặc biệt trong một số API network; vì vậy khi dữ liệu cần phân biệt “không được cấu hình” với số 0, ta cần một biểu diễn khác như pointer, boolean riêng hoặc type có ý nghĩa rõ ràng.

Tóm lại

`package main` và `func main` là entry point theo quy ước của executable. `go run .` chạy package hiện tại; `go build` tạo artifact. Type inference vẫn là static typing, còn zero value là một tính năng thiết kế có ích nhưng cần được đọc trong ngữ cảnh. Chương tiếp theo sẽ dùng các kiểu và control flow để biến một chương trình chỉ in text thành một chương trình có quyết định rõ ràng.

Nguồn

- Go Language Specification: Packages, Program initialization and execution, Variables.
- Effective Go: Formatting; Control structures.
- Go command documentation, `go.dev/cmd/go`.